Skills & Agents in Large Language Models

From Prompted Reasoning to Autonomous Architectures

Prof. D.Sc. BARSEKH-ONJI Aboud

Facultad de Ingeniería
Universidad Anáhuac México

April 6, 2026

Table of Contents

Agenda

The Evolution of Language Models

Generation 1: Pure Language Models

- Input → text output only
- No memory beyond context window
- No external actions
- GPT-2 (2019), BERT (2018)

Generation 2: Instruction-Tuned Models

- RLHF alignment
- Follow complex instructions
- GPT-3.5, Claude 1/2, Llama 2

Generation 3: Tool-Augmented Models

- Function calling / tool use
- Read files, call APIs, run code
- GPT-4, Claude 3, Gemini

Generation 4: Agentic Models

- Autonomous multi-step reasoning
- Spawn sub-agents, delegate tasks
- Plan → Act → Observe loops
- Claude 3.5/4, GPT-4o, Gemini 2.0

What Makes a System "Agentic"?

Core Properties of an Agent

An **agent** is any system that perceives its environment and takes actions to achieve a goal. For LLM-based agents, we add:

1. Perception

Reads text, files, API responses, tool outputs, memory stores

2. Reasoning

Plans steps, decides which tools to call, evaluates partial results

3. Action

Executes tools, writes files, calls APIs, spawns sub-agents, sends messages

Key Insight

The LLM is the *reasoning engine*. Tools, memory, and orchestration are the *scaffolding*. The combination produces agentic behavior.

The ReAct Framework: Reasoning + Acting

ReAct Loop (Yao et al., 2022)

1. **Thought:** the model reasons about what to do next
2. **Action:** the model calls a tool or function
3. **Observation:** the result is fed back into context
4. Repeat until goal is achieved

This loop is the backbone of every modern agentic framework:

Example ReAct Trace

```
Task: What is the current BTC price in MXN?

Thought: I need to look up the current price.
I'll use the web search tool.

Action: WebSearch({''query'': ''BTC price USD''})

Observation: BTC = $62,400 USD

Thought: Now I need the USD/MXN rate.

Action: WebSearch({''query'': ''USD MXN exchange
rate''})

Observation: 1 USD = 17.20 MXN

Thought: I can compute: 62400 × 17.20
```

Agenda

Tool Use: The Interface Between LLM and World

What is a Tool?

A **tool** (also called *function*) is a structured capability that the LLM can invoke by generating a JSON-formatted call. The host application executes it and returns the result.

Tool definition (sent in system prompt):

- Name and description (natural language)
- Input schema (JSON Schema)
- Output format

At runtime:

Tool Schema Example

```
{
  "name": "read_file",
  "description": "Read the contents of
      a file at the given path",
  "input_schema": {
    "type": "object",
    "properties": {
      "path": {
```

How Claude Uses Tools Internally

Multi-Turn Tool Loop

1. User sends message + tools list
2. Claude generates `tool_use` block
3. Host application executes tool
4. Result sent as `tool_result` message
5. Claude continues reasoning
6. Steps 2–5 repeat as needed

Important: The LLM Never Executes Code

Claude *generates* structured JSON requests. Your Python/JS/shell code *actually runs* the tools. This separation is fundamental to safety — you control what tools exist and what they can do.

Anthropic API Tool Loop

```
# Simplified loop
while True:
    resp = client.messages.create(...)
    if resp.stop_reason == "tool_use":
```

Agenda

What is a Skill in Claude Code?

Definition

A **skill** is a reusable, self-contained **prompt template** (written in Markdown) that extends Claude Code with a custom slash command. When invoked, the skill's Markdown content is injected into the conversation as a system-level instruction set.

- Lives in `~/.claude/skills/` (user-level) or `.claude/skills/` (project-level)
- Each skill is a `.md` file
- Invoked with `/<skill-name>`
- No separate process or runtime needed
- Claude *reads* the skill and acts

Skill Directory Structure

```
~/.claude/
  skills/
    commit.md            # /commit
    review-pr.md         # /review-pr
    latex-academic.md    # /latex-
        academic
    beamer-slides.md     # /beamer-slides
    nodos-newsletter.md
```

Anatomy of a Skill File

Required Elements

1. **Name** (filename defines the slash command)
2. **Trigger description** (when to invoke)
3. **Context/persona** (role, conventions)
4. **Step-by-step instructions**
5. **Output format** (what to produce)
6. **Examples** (optional but powerful)

Minimal Skill: `summarize.md`

```
# Summarize Skill

## Trigger
When the user wants a bullet-point
summary of a document or file.

## Instructions
1. Read the provided file or text.
2. Identify the 5 most important ideas.
3. Write each idea as a concise
   bullet (max 15 words).
4. Add a one-sentence TL;DR at top.

## Output Format
**TL;DR:** <one sentence>
```

Skills: Capabilities and Limitations

What Skills CAN Do

- Encode complex, multi-step workflows
- Define consistent output formats
- Set domain-specific conventions (LaTeX style, code standards)
- Inject expertise on demand (e.g., "act as an exam writer")
- Chain naturally with tool use (Claude still uses tools during a skill)

What Skills CANNOT Do

- Execute code autonomously
- Spawn sub-processes or sub-agents
- Run on a schedule or trigger from events
- Maintain state between invocations
- Communicate with external systems on their own
- Replace an agent for multi-step autonomous tasks

Real-World Skill Example: `latex-academic`

This skill is used in this course. When invoked with `/latex-academic`, it:

1. Loads author information (Prof. Dr. Barsekh-Onji, Anáhuac México)
2. Selects the appropriate document template (exam, letter, paper, syllabus)
3. Applies university LaTeX conventions (margins, fonts, booktabs tables)
4. Compiles with `pdflatex` twice for correct TOC/refs
5. Delivers both `.tex` source and

What Makes It Powerful

Without the skill, you would need to repeat in every session:

- Author name and credentials
- Institution name and email
- Preferred packages and margins
- Compilation workflow
- Table and code listing styles

The skill encodes all of this *once*, making it instantly reusable.

Agenda

What is an Agent in Claude Code?

Definition

An **agent** in Claude Code is an **autonomous subprocess** — a separate Claude instance with its own context, tools, and instructions — that Claude (the orchestrator) can launch to execute complex, multi-step tasks independently and in parallel.

- Defined in `.claude/agents/` as `.md` files
- Has a **system prompt** (its personality and expertise)
- Has an explicit **tools list** (what it can do)

Key Difference from Skills

A skill is a prompt *injected into the current conversation*. An agent is a *separate Claude instance* with its own lifecycle, context, and tools.

The Agent Tool

Anatomy of an Agent Definition File

Agent File Structure

Each agent is a `.md` file with a YAML frontmatter block followed by the system prompt body:

Frontmatter fields:

- `name` — identifier used by the orchestrator
- `description` — when to use this

Agent File Template

```
---
name: my-agent
description: >
  Use this agent when the user needs
  X. Triggers: "do X", "help with Y".
tools:
  - Read
  - Write
  - Bash
  - WebSearch
model: claude-sonnet-4-6
---

# My Agent
```

How the Orchestrator Launches an Agent

The Agent Tool Call

The main Claude instance calls the `Agent` tool with:

- `subagent_type`: the agent name from frontmatter
- `prompt`: task description (full context)
- `description`: short label (3–5 words)
- `run_in_background`: true/false
- `isolation`: "worktree" for git isolation

Orchestrator Spawning an Agent

```
// What Claude generates internally:
{
  "tool": "Agent",
  "input": {
    "subagent_type": "code-reviewer",
    "description": "Review
        authentication module",
    "prompt": "Review the file src/auth
        .py
      for security issues. Focus on:
      1. SQL injection risks
      2. Session token handling
      3. Password storage
      Return findings as a list."
```

Agents: Tool Scoping and Isolation

Tool Scoping (Security)

Each agent only has access to the tools listed in its `tools:` frontmatter. This is a critical safety feature:

- A **read-only agent** gets only `Read`, `Grep`, `Glob`
- A **research agent** gets `WebSearch`, `WebFetch`, `Read`
- A **code-writer agent** gets `Read`, `Write`, `Edit`, `Bash`
- A **deployment agent** gets `Bash`

Git Worktree Isolation

When `isolation: "worktree"` is set, the agent works on a **temporary branch copy** of the repository. Useful for:

- Parallel feature development
- Experimental changes (rollback-safe)
- Code review without touching main branch

Context Isolation

Agents **do not share** the orchestrator's

Parallel Agent Execution

Parallel vs Sequential Agents

Claude Code can launch multiple agents simultaneously when tasks are independent. This is a powerful technique for large codebases.

Sequential (slow)

1. Agent A: search for bug in module 1 *(done)*
2. Agent B: search for bug in module 2 *(done)*
3. Agent C: search for bug in module 3 *(done)*

Parallel (fast)

- Agent A, B, C launched simultaneously
- All run in parallel
- Orchestrator waits for all
- Synthesizes results

Agenda

Skills vs Agents: Core Distinctions

Dimension	Skill	Agent
Nature	Prompt template	Autonomous subprocess
Invocation	`/skill-name` by user	`Agent` tool by Claude
Execution context	Current conversation	New isolated conversation
Memory	Shared with main session	Independent (starts cold)
Tool access	Same as main session	Explicitly scoped subset
Parallelism	No (synchronous)	Yes (background support)
Lifecycle	Ends with response	Has own task lifecycle
Defined by	`.md` in `skills/`	`.md` in `agents/`
Spawner	User	Claude (orchestrator)
Code execution	Via Claude's tools	Via agent's scoped tools

Skills vs Agents: When to Use Which

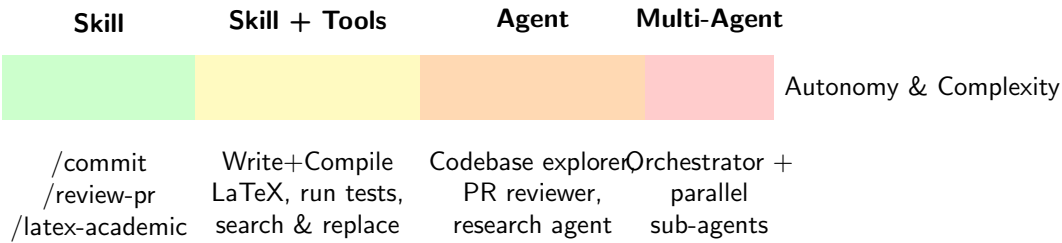
Use a SKILL when...

- The task is **well-scoped** and short
- You want **consistent behavior** for a recurring activity (compile LaTeX, write a commit message, generate an exam)
- You need **domain conventions** injected (LaTeX style, code standards)
- The user explicitly triggers the workflow
- **No parallel execution** is needed

Use an AGENT when...

- The task is **long, multi-step**, or exploratory
- You need to search a **large codebase** in parallel
- The subtask is **independent** of the main conversation
- You want **isolated context** (no contamination of main session)
- The subtask involves **risky tools** that should be scoped

The Skill–Agent Continuum



Design Principle

Always use the *simplest* mechanism that solves the problem. Agents add power but also complexity, cost, and latency. Start with a skill; escalate to an agent when needed.

Agenda

Multi-Agent System Topologies

1. Orchestrator–Worker

One main Claude instance manages multiple specialized workers. Workers report back; orchestrator synthesizes.

- Best for: parallel exploration, specialized tasks
- Example: research orchestrator spawning domain-specific search agents

2. Pipeline (Sequential)

3. Peer-to-Peer / Debate

Multiple agents work on the same problem and argue/critique each other's output. The orchestrator picks the best solution.

- Best for: code review, adversarial testing, writing quality

Claude Code's Native Topology

Claude Code uses the **Orchestrator–Worker** pattern. The main

Context Management in Multi-Agent Systems

The Context Window Problem

Every LLM has a finite context window (Claude: 200k tokens). Long agentic tasks can exhaust it. Multi-agent systems solve this by **distributing context**.

- Each agent has its own context window
- Agents read only what they need
- Results are **compressed** before returning to orchestrator
- The orchestrator's context stays clean

Example: Large Codebase Analysis

Agent Memory: What Persists?

- **Within a turn:** agent reads files, calls tools, builds a result
- **Between turns:** nothing — agents start cold each time
- **Persistent state:** write to files,

Agenda

The Anthropic Python SDK: Foundations

Installation

```
# Always use conda env 'research'
conda activate research
pip install anthropic

# Verify
python -c "import anthropic;
           print(anthropic.__version__)
           "
```

API Key Setup

```
# Add to ~/.zshrc or ~/.bashrc
export ANTHROPIC_API_KEY="sk-ant-..."
```

Basic API Call

```
import anthropic

client = anthropic.Anthropic()

message = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[
        {
            "role": "user",
            "content": "Explain the
              perceptron algorithm."
        }
    ]
)
```

Building a Tool-Using Agent: Full Example

```python
import anthropic, json

client = anthropic.Anthropic()

# 1. Define tools
tools = [
    {
        "name": "get_course_grade",
        "description": "Look up a student's grade for a given course.",
        "input_schema": {
            "type": "object",
            "properties": {
                "student_id": {"type": "string", "description": "Student ID (e.g. A00123456)"},
                "course":     {"type": "string", "description": "Course name"}
            },
            "required": ["student_id", "course"]
        }
```

Building a Tool-Using Agent: The ReAct Loop

```python
# 3. Agent loop with tool execution
def run_agent(user_query: str):
    messages = [{"role": "user", "content": user_query}]

    while True:
        response = client.messages.create(
            model="claude-sonnet-4-6",
            max_tokens=1024,
            tools=tools,
            messages=messages
        )

        # Add assistant response to history
        messages.append({"role": "assistant", "content": response.content})

        if response.stop_reason == "end_turn":
            # Extract final text
            return next(b.text for b in response.content if b.type == "text")
```

Streaming Agent Responses

Why Streaming?

Agentic tasks can be slow. Streaming lets you show partial results in real time, improving user experience dramatically.

```python
# Streaming with context manager
with client.messages.stream(
    model="claude-sonnet-4-6",
    max_tokens=2048,
    messages=messages,
    tools=tools,
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
```

Streaming Events

- `message_start` — metadata
- `content_block_start` — new text or tool block
- `content_block_delta` — incremental text
- `content_block_stop` — block complete
- `message_stop` — full response done

Best Practice

Agenda

Creating a Custom Agent for Claude Code

Step 1: Create the Agent Directory

```
# Project-level agents (only this project)
mkdir -p .claude/agents/

# User-level agents (all projects)
mkdir -p ~/.claude/agents/
```

Step 2: Create the Agent File

Create `.claude/agents/my-agent.md` with:

1. YAML frontmatter (name, description, tools)

Common Mistakes

- Vague description → agent never gets selected
- Too many tools → security surface grows

Agent Routing: How Claude Chooses the Right Agent

The Selection Algorithm (Simplified)

When Claude (orchestrator) decides to delegate a task, it reads the `description` field of all available agents and selects the best match based on semantic similarity to the current task.

Bad Description (vague)

```
---
name: helper
description: Helps with things.
tools: [Read, Bash]
---
```

Good Description (specific)

```
---
name: matlab-runner
description: >
  Use this agent to run, debug, or
  analyze MATLAB scripts (.m files).
  Triggers: "run this MATLAB script",
  "debug my .m file", "MATLAB error",
  "plot this in MATLAB".
```

Agenda

Demo Agent: `neural-net-tutor`

Objective

We will create a complete agent for this course: `neural-net-tutor`. This agent answers conceptual questions about neural networks, explains architectures, and generates MATLAB code examples. It has **read-only** access (no file modification).

Agent Capabilities:

- Explain NN concepts at undergraduate level
- Generate MATLAB Deep Learning Toolbox examples
- Reference course materials (reads `.tex` and `.m` files)

Safety Design

The agent has **no Write, Edit, or Bash tools**. It can only read — it cannot modify course materials, run commands, or access the internet. This is intentional: a tutor should advise, not act.

Demo Agent: File Content

```
---
name: neural-net-tutor
description: >
  Use this agent to explain neural network concepts, architectures,
  and training algorithms. Triggers: "explain backpropagation",
  "how does LSTM work", "what is a CNN", "show me a MATLAB example",
  "help me understand [NN topic]", any conceptual NN/SVM question.
tools:
  - Read
  - Grep
  - Glob
model: claude-sonnet-4-6
---

# Neural Network Tutor

You are an expert teaching assistant for the course *Redes Neuronales y SVM*
at Universidad Anahuac Mexico, taught by Prof. Dr. Aboud Barsekh-Onji.
```

Demo Agent: System Prompt (continued)

```
## Behavior Rules
1. Always start with an intuitive explanation before the math
2. Use Spanish technical terminology with English equivalents in parentheses
3. When generating MATLAB code, always use:
   - trainnet() (R2023a+ API, never legacy train())
   - trainingOptions("adam", ...) with named arguments
   - minibatchpredict() + scores2label() for evaluation
4. Keep examples self-contained and runnable
5. If a concept is in the course slides, mention the slide file name

## Knowledge Scope
- Feedforward Neural Networks (perceptron, MLP, backpropagation)
- Convolutional Neural Networks (LeNet, AlexNet, ResNet)
- Recurrent Networks (RNN, LSTM, GRU)
- Support Vector Machines (kernel trick, soft margin)
- Transfer Learning (feature extraction, fine-tuning)
- Optimization (SGD, Adam, learning rate schedules)

```

Demo: Using the Agent

User asks Claude Code:

"Explain the vanishing gradient problem and how LSTMs solve it. Show me a MATLAB example."

What happens internally:

1. Claude (orchestrator) reads task
2. Matches task to `neural-net-tutor` (NN concept question)
3. Launches agent with task as prompt
4. Agent reads course files via `Glob`/`Grep`

Agent Response (excerpt)

Vanishing Gradient — Intuition: During backpropagation, gradients are multiplied by the weight matrix at each layer. If weights < 1, this product shrinks exponentially:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \prod_{l=2}^{L} \frac{\partial h^{(l)}}{\partial h^{(l-1)}} \cdot \frac{\partial \mathcal{L}}{\partial h^{(L)}}$$

For deep networks, early layers receive gradients ≈ 0 — they stop learning.

Agenda

Lab Activity: Build Your Own Agent

Objective

Create a functional Claude Code agent for a domain you know well. Test it by asking it three different questions.

1. **Choose a domain:**
   - MATLAB script debugger
   - Python data analysis assistant
   - Literature review summarizer
   - LaTeX equation helper
   - SVM hyperparameter advisor
2. **Create** `.claude/agents/your-agent.md`
5. **Test** by asking Claude Code a question your agent should handle
6. **Verify** the agent was invoked (look for Agent tool call in output)
7. **Iterate**: refine the description or system prompt based on results
8. **Deliverable**: submit your `.md` file with a 1-page reflection on design decisions

Summary

Key Concepts Covered

- LLMs evolved from text generators to agentic systems via tool use
- The ReAct loop (Thought $\rightarrow$ Act $\rightarrow$ Observe) is the core pattern
- **Skills** = reusable prompt templates, invoked by user via `/command`
- **Agents** = autonomous subprocesses with scoped tools, invoked by Claude
- Skills for: consistent workflows

What You Can Now Do

- Write a skill file for any recurring workflow
- Write an agent definition for any specialized task
- Build a tool-using agent with the Anthropic Python SDK
- Design multi-agent orchestration for complex problems
- Scope tools appropriately for security
- Write effective agent descriptions for

References and Further Reading

1. **Yao, S. et al.** (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023. https://arxiv.org/abs/2210.03629
2. **Anthropic.** (2025). *Claude API Documentation — Tool Use*. https://docs.anthropic.com/en/docs/tool-use
3. **Anthropic.** (2025). *Claude Code — Agents and Skills*. https://docs.anthropic.com/en/docs/claude-code
4. **Wang, L. et al.** (2024). *A Survey on Large Language Model based Autonomous Agents*. Frontiers of Computer Science, 18(6).
5. **Xi, Z. et al.** (2023). *The Rise and Potential of Large Language Model Based Agents: A Survey*. https://arxiv.org/abs/2309.07864
6. **Anthropic.** (2025). *Anthropic Python SDK Reference*. https://github.com/anthropics/anthropic-sdk-python